



รายงาน

PS03: Process Manipulation and Monitoring

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ข้อ 1: คำสั่ง Linux สำหรับจัดการและเฝ้าดู process และ job	1
1.1 ความแตกต่างระหว่าง top และ ps	1
1.1.1 ผลการทดลอง	2
1.1.2 อภิปรายผล	3
1.2 จุดประสงค์ของคำสั่ง nice และวิธี nice process	3
1.2.1 วิธีวัดผล	3
1.2.2 อภิปรายผล	4
1.3 การ kill process และ job	5
1.3.1 ผลการทดลอง	5
1.3.2 อภิปรายผล	7
2 ข้อ 2: การรัน shell script แบบ foreground / background และ job control	7
2.1 สถานการณ์ที่ 1: ./ss1_1.sh เทียบกับ ./ss1_1.sh &	7
2.1.1 อภิปรายผล	9
2.2 สถานการณ์ที่ 2: CTRL+Z, jobs, fg และ bg	9
2.2.1 อภิปรายผล	11
3 ข้อ 5: ฟังก์ชันภาษาซีจำลองการทำงานของ STCF	12
3.1 Source code	12
3.2 ผลการทดลองและการตรวจสอบความถูกต้อง	13
3.3 อภิปรายผล	15

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11), kernel 6.6.87.2-microsoft-standard-WSL2
จำนวน CPU	22 logical CPUs (nproc)
เครื่องมือ	procs-ng 4.0.4 (ps, top, kill, pgrep, pkill), util-linux (renice, taskset), psmisc (killall)
Shell	GNU bash 5.2.21(1)-release
คอมไพเลอร์	gcc 13.3.0 คอมไพล์ด้วย -Wall -Wextra ผ่านทุกไฟล์ ไม่มี warning
วันที่เก็บผล	31 สิงหาคม 2569

ผลการทดลองดิบ ทุกบรรทัดที่ยกมาในรายงานนี้ตัดมาจากไฟล์ `result/q1_verified.txt`, `result/q2_verified.txt` และ `result/q5_verified.txt` ซึ่งสร้างซ้ำได้ด้วย `make result` (สคริปต์ `run_q1.sh`, `run_q2.sh`, `run_q5.sh` ในโฟลเดอร์ `src`) ในรายงานแสดงเฉพาะบรรทัดที่จำเป็นต่อการอภิปราย ส่วนที่ตัดออกจะมีเครื่องหมาย ... หรือหมายเหตุ (lines omitted) กำกับไว้

หมายเหตุสองเรื่องเกี่ยวกับวิธีทดลอง (1) เครื่องที่ใช้มี 22 CPU ถ้าปล่อยให้โปรแกรมทดสอบวิ่งอิสระ ทุก process จะได้ CPU คนละแกนและจะ *ไม่เห็น* ผลของค่า `nice` เลย การทดลองข้อ 1.2 จึงใช้ `taskset -c 0` บังคับให้ทุก process ทดสอบอยู่บน CPU แกนเดียวกัน เพื่อสร้างการแย่ง CPU ซึ่งเป็นเงื่อนไขเดียวกับที่ `nice` มีความหมาย (2) CTRL+Z และคำสั่ง `jobs/fg/bg` ต้องการ shell แบบ interactive ที่มี tty จริง การเก็บผลข้อ 2 จึงใช้ `script(1)` สร้าง pseudo terminal แล้วป้อนคำสั่งที่ละบรรทัดเหมือนคนพิมพ์ ส่วน CTRL+Z ส่งเป็นไบนารี `0x1A` ให้ line discipline ของ tty แปลงเป็น SIGTSTP เอง transcript ที่ได้จึงเป็นของ terminal จริง ไม่ใช้การจำลอง

1 ข้อ 1: คำสั่ง Linux สำหรับจัดการและเฝ้าดู process และ job

1.1 ความแตกต่างระหว่าง top และ ps

ทั้งสองคำสั่งอ่านข้อมูลจากแหล่งเดียวกันคือ `/proc` ความต่างจึงไม่ได้อยู่ที่ *ข้อมูล* แต่อยู่ที่ *วิธีนำเสนอ* `ps` เป็นคำสั่งแบบ one-shot คืออ่าน `/proc` รอบเดียว พิมพ์ผล แล้วจบทันที ส่วน `top` เป็นโปรแกรม interactive ที่วนอ่านซ้ำทุกช่วงเวลา (ตั้งต้น 3 วินาที) แล้ววาดหน้าจอใหม่ทับของเดิม จนกว่าผู้ใช้จะกด q

Table 1: สรุปความแตกต่างระหว่าง ps และ top

ประเด็น	ps	top
รูปแบบการทำงาน	one-shot อ่านครั้งเดียวแล้วจบ	วนอ่านซ้ำและวาดหน้าจอใหม่เรื่อย ๆ
ความหมาย %CPU	ค่าเฉลี่ยตลอดอายุ process (เวลา CPU ที่ใช้ทั้งหมดหารด้วยอายุ)	ค่าเฉพาะช่วงรีเฟรชรอบล่าสุด จึงสะท้อนสถานการณ์ปัจจุบัน
ภาพรวมของระบบ	ไม่มี ต้องหาเอง	มีส่วนหัว ทั้ง load average, จำนวน task แยกสถานะ, %Cpu(s) และหน่วยความจำ
การโต้ตอบ	ไม่มี	มี กด k เพื่อ kill, r เพื่อ renice, M/P เพื่อเรียงตาม memory/CPU
นำผลไปใช้ต่อ	เหมาะมาก ต่อ pipe หรือเขียนลงไฟล์ในสคริปต์ได้ทันที	ต้องใช้ -b (batch) ก่อน จึงจะนำไปใช้ต่อได้
เลือกคอลัมน์	ยืดหยุ่นสูงด้วย -o และเรียงด้วย --sort	ได้เฉพาะชุดที่โปรแกรมเตรียมไว้
สถานการณ์ที่เหมาะสม	ต้องการ <i>ค้นหา</i> process หรือเก็บข้อมูลเข้าสคริปต์	ต้องการ <i>เฝ้าดู</i> ว่าตอนนี้ใครกิน CPU หรือ RAM มากที่สุด

1.1.1 ผลการทดลอง

ps เปล่า ๆ เห็นเฉพาะ process ของผู้ใช้เดียวกันที่ผูกกับ terminal เดียวกัน การจะเห็นทั้งระบบต้องใส่ option ซึ่งมีสองไวยากรณ์ คือแบบ BSD (aux) ที่ให้ %CPU, %MEM, STAT และแบบ UNIX (-ef) ที่ให้คอลัมน์ PPID สำหรับไล่ความสัมพันธ์พ่อลูก

```
$ ps
  PID TTY          TIME CMD
 5688 pts/4    00:00:00 bash
 5689 pts/4    00:00:00 bash
 5691 pts/4    00:00:00 sed
 5744 pts/4    00:00:00 ps

$ ps aux | head -8
USER            PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root              1  0.0  0.0 21792 12084 ?        Ss   12:03   0:01 /sbin/init
root              2  0.0  0.0  3072  1760 ?        S1   12:03   0:00 /init
root              7  0.0  0.0  3132  2112 ?        S1   12:03   0:07 plan9 --control-socket 7 --log-level 4 --server-fd 8

$ ps -ef | head -8
UID            PID  PPID  C  STIME TTY          TIME CMD
root              1      0  0  12:03 ?        00:00:01 /sbin/init
root              2      1  0  12:03 ?        00:00:00 /init
root              7      2  0  12:03 ?        00:00:07 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --
```

Code 1: ps เปล่า ๆ เทียบกับ ps aux และ ps -ef (ตัดมาบางบรรทัด)

จุดแข็งที่สุดของ ps คือ -o ที่เลือกคอลัมน์เองได้และ --sort ที่เรียงลำดับได้ ในการทดลองนี้สั่ง q1_burn ให้กิน CPU ค้างไว้เป็นเป้าก่อน แล้วจึงเรียงตาม %CPU และเนื่องจาก ps จบพื้นที่ จึงวัดเวลาได้ต่ำกว่า 0.01 วินาที

```
$ taskset -c 0 ./q1_burn 30 MONITOR_ME & # a CPU-hungry process to watch
$ ps -eo pid,ppid,ni,pri,stat,%cpu,%mem,etime,comm --sort=-%cpu | head -6
  PID  PPID  NI  PRI  STAT %CPU %MEM    ELAPSED COMMAND
 5749   5689   0   19  R+    109  0.0      00:02 q1_burn
      7      2   0   19  S1     0.0  0.0      02:51:41 init
  102     1   0   19  Ss     0.0  0.0      02:51:41 systemd-udev

$ /usr/bin/time -f "real %e s" ps aux > /dev/null
real 0.00 s
$ ps -e --no-headers | wc -l
39
```

Code 2: ps -eo ... --sort และการวัดว่า ps จบพื้นที่

top ต้องใช้ -b คู่กับ -n จึงจะเก็บผลลงรายงานได้ ห้าบรรทัดแรกคือส่วนสรุปภาพรวมของระบบซึ่ง ps ไม่มีให้เลย

```
$ top -b -n 1 | head -13
top - 14:55:11 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.5 us, 0.0 sy, 0.0 ni, 95.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15716.6 total, 13911.9 free, 1368.4 used, 694.1 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used. 14348.3 avail Mem

   PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 5749 poomipat  20   0   2548  1056  1056 R 100.0   0.0   0:02.43 q1_burn
      1 root      20   0 21792 12084 9092 S   0.0   0.1   0:01.89 systemd
      2 root      20   0  3072  1760  1760 S   0.0   0.0   0:00.10 init-systemd(Ub
```

Code 3: top -b -n 1 (ตัดมา 3 แถวแรก)

```
$ top -b -n 2 -d 1 | grep -E "^top -|^Tasks|^%Cpu|q1_burn"
top - 14:55:11 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.4 us, 0.0 sy, 0.0 ni, 95.6 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
 5749 poomipat  20   0   2548  1056  1056 R 100.0   0.0   0:02.66 q1_burn
top - 14:55:12 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.5 us, 0.0 sy, 0.0 ni, 95.4 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
 5749 poomipat  20   0   2548  1056  1056 R 100.0   0.0   0:03.76 q1_burn
```

Code 4: top -b -n 2 -d 1 สองภาพห่างกัน 1 วินาที

1.1.2 อภิปรายผล

- **ต่างกันที่มีเวลาเป็นหลัก:** ps ตอบว่า “ณ วินาทีนี้มีอะไรอยู่ในระบบ” ส่วน top ตอบว่า “ตอนนี้ระบบกำลังทำอะไรอยู่” ยืนยันได้จากการที่ ps aux ใช้เวลา 0.00 วินาทีแล้วจบ ในขณะที่ top ต้องถูกบังคับด้วย -n 1 ถึงจะยอมจบ
- **ค่า %CPU ของสองคำสั่งไม่ใช่ค่าเดียวกัน:** เป็นจุดที่สับสนบ่อยที่สุด ของ ps คือเวลา CPU ที่ใช้ทั้งหมดหารด้วยอายุ process จึงเป็นค่าเฉลี่ย ย้อนหลัง ส่วนของ top คิดจากส่วนต่างระหว่างสองรอบรีเฟรชล่าสุด ผลคือ process ที่เคยกิน CPU หนักเมื่อชั่วโมงก่อนแต่ตอนนี้หลับอยู่ จะยังมี %CPU สูงใน ps แต่เป็น 0.0 ใน top ในการทดลองนี้ ps รายงาน 109 ให้ q1_burn ที่เพิ่งเกิดได้ 2 วินาที (คอลัมน์ ELAPSED บัดเป็นวินาทีเต็ม จึงหารแล้วเกิน 100 เล็กน้อย) ขณะที่ top รายงาน 100.0 ตรงตามจริง
- **top ให้ภาพรวมของระบบ ส่วน ps ให้ภาพของ process:** load average, จำนวน zombie, %Cpu(s) แยก us/sy/ni/id/wa และหน่วยความจำรวม มีเฉพาะใน top ถ้าจะเอาจาก ps ต้องไปอ่าน /proc/loadavg และ /proc/meminfo เอง นอกจากนี้ผลการทดลองยังยืนยันว่า top คำนวณใหม่ทุกรอบจริง คือ TIME+ ของ q1_burn เพิ่มจาก 0:02.66 เป็น 0:03.76 คือ 1.10 วินาทีในเวลาจริง 1 วินาที ตรงกับที่รายงาน %CPU เป็น 100.0 พอดี
- **แต่ละคำสั่งทำสิ่งที่อีกคำสั่งทำไม่ได้:** ps ชนะขาดเรื่องการนำผล ไปใช้ต่อ เพราะต่อ pipe ได้ทันทีและเลือกคอลัมน์เองได้ (ใช้ตลอดทั้งรายงานนี้) ส่วน top “สั่งการ” ได้ในตัว กด k ส่ง signal หรือ r เพื่อ renice ได้โดยไม่ต้องออกจากโปรแกรม จึงครบวงจรกว่าเมื่อต้องแก้ปัญหาเฉพาะหน้า

1.2 จุดประสงค์ของคำสั่ง nice และวิธี nice process

nice ใช้ บอกใบ้ให้ scheduler ทราบว่า process ตัวนี้สำคัญมากหรือน้อย เมื่อเทียบกับ process อื่นที่แย่ง CPU กันอยู่ ผ่านตัวเลข nice value ช่วง -20 ถึง +19 ค่าตั้งต้นคือ 0 ค่ายิ่งมากยิ่ง “ใจดี” คือยอมสละ CPU ให้คนอื่นก่อน ตัวเองจึงได้ priority ต่ำ ส่วนค่าติดลบคือแย่ง CPU มาใช้ก่อน จุดที่ต้องเน้นคือ nice ไม่ใช้การจองหรือจำกัดโควตา CPU ถ้าไม่มีใครแย่ง process ที่ nice เป็น 19 ก็ยังใช้ CPU ได้เต็ม 100% การใช้งานจริงคือสั่งงานหนักที่ไม่รีบ เช่น backup หรือ compile โปรแกรมใหญ่ ให้ทำงานด้วยค่า nice สูง ๆ เพื่อไม่ให้ไปแย่ง CPU จนหน้าจของผู้ใช้กระตุก

วิธี nice process มีสองกรณีตามจังหวะเวลา คือ (1) กำหนดตั้งแต่เริ่มโปรแกรม ด้วย nice -n <value> <command> และ (2) เปลี่ยนของ process ที่รันอยู่แล้วด้วย renice -n <value> -p <PID> ซึ่งยังระบุเป็นรายผู้ใช้ (-u) หรือรายกลุ่ม (-g) ได้ด้วย นอกจากนี้ยังเปลี่ยนได้จากหน้าจอ top โดยกด r ซึ่งเบื้องหลังก็คือ system call setpriority() เหมือนกัน ส่วน **ข้อจำกัดด้านสิทธิ์** คือผู้ใช้ทั่วไป เพิ่ม ค่า nice ได้เสมอ แต่ ลด ไม่ได้เลย แม้จะเป็นการลดกลับมาที่ค่าเดิมของ process ตัวเอง การลดค่าต้องเป็น root หรือมีสิทธิ์ CAP_SYS_NICE

1.2.1 วิธีวัดผล

เขียนโปรแกรม q1_burn.c ที่วนคำนวณกิน CPU ตามเวลาที่กำหนด แล้วรายงานว่า ทำงานได้กี่ “รอบ” (คอมไพล์ด้วย -o0 เพื่อไม่ให้คอมไพเลอร์ดัดลุบทั้ง) เมื่อรันสองตัวพร้อมกันบน CPU แยกเดี่ยว อัตราส่วนของจำนวนรอบก็คืออัตราส่วนของ CPU ที่ scheduler แบ่งให้โดยตรง และมี *กรณีควบคุม* ที่ตั้งค่า nice เท่ากันไว้เทียบด้วย เพื่อให้มั่นใจว่าความต่างที่เห็นมาจากค่า nice จริง ๆ

```
$ nice -> 0
$ nice -n 10 nice -> 10
$ nice -n 19 nice -> 19
$ nice -n 5 nice -n 5 nice (increments add up) -> 10
```

Code 5: ค่า nice ตั้งต้น และการที่ nice เป็นการเพิ่มค่าสะสม ไม่ใช่กำหนดค่าตายตัว

```
$ taskset -c 0 ./q1_burn 15 A_nice0 &
$ taskset -c 0 ./q1_burn 15 B_nice0 &
$ ps -o pid,ni,pri,psr,stat,%cpu,time,args -p <pid1>,<pid2> # while both are running
PID NI PRI PSR STAT %CPU TIME COMMAND
```

```

5765  0 19  0 R+  55.3 00:00:02 /tmp/ps03_q1/q1_burn 15 A_nice0
5766  0 19  0 R+  55.5 00:00:02 /tmp/ps03_q1/q1_burn 15 B_nice0
[A_nice0] pid=5765 nice=0 rounds=19068 wall=15.00s
[B_nice0] pid=5766 nice=0 rounds=19119 wall=15.00s

$ taskset -c 0          ./q1_burn 15 HIGH_nice0 &
$ taskset -c 0 nice -n 19 ./q1_burn 15 LOW_nice19 &
$ ps -o pid,ni,pri,psr,stat,%cpu,time,args -p <pid1>,<pid2>      # while both are running
  PID NI PRI PSR STAT %CPU   TIME COMMAND
  5771  0 19  0 R+   106 00:00:04 /tmp/ps03_q1/q1_burn 15 HIGH_nice0
  5772 19  0  0 RN+    1.7 00:00:00 /tmp/ps03_q1/q1_burn 15 LOW_nice19

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
  5771 poomipat   20   0   2548    1056    1056 R   100.0   0.0   0:04.48 q1_burn
  5772 poomipat   39  19   2548    1056    1056 R    0.0   0.0   0:00.07 q1_burn
[HIGH_nice0] pid=5771 nice=0 rounds=29924 wall=15.00s
[LOW_nice19] pid=5772 nice=19 rounds=465 wall=15.00s

```

Code 6: กรณีควบคุม (nice 0 กับ 0) เทียบกับกรณีทดลอง (nice 0 กับ 19) ผู้ก๊วที่ CPU 0 ทั้งคู่

```

$ ps -o pid,ni,pri,stat,comm -p <pid>      # before renice
  PID NI PRI STAT COMMAND
  5777  0 19 R+   q1_burn
$ renice -n 15 -p 5777
5777 (process ID) old priority 0, new priority 15
$ ps -o pid,ni,pri,stat,comm -p <pid>      # after renice
  PID NI PRI STAT COMMAND
  5777 15  4 RN+   q1_burn
[renice_target] pid=5777 nice=15 rounds=30658 wall=12.00s

```

Code 7: renice เปลี่ยนค่าของ process ที่กำลังรันอยู่

```

$ nice -n -5 nice
nice: cannot set niceness: Permission denied
0

$ renice -n -5 -p 5785
renice: failed to set priority for 5785 (process ID): Permission denied
exit status = 1

$ renice -n +8 -p 5785      # raising the nice value is always allowed
5785 (process ID) old priority 0, new priority 8
  PID NI PRI STAT COMMAND
  5785  8 11 SN+  sleep

$ renice -n 0 -p 5785      # lowering it back is refused, even for our own process
renice: failed to set priority for 5785 (process ID): Permission denied

```

Code 8: ผู้ใช้ทั่วไปเพิ่มค่า nice ได้ แต่ลดไม่ได้

1.2.2 อภิปรายผล

Table 2: ผลของค่า nice ต่อการแบ่ง CPU (ทุกกรณีถูกผู้ก๊วที่ CPU 0)

การทดลอง	nice	รอบที่ทำได้	รอบ/วินาที	สัดส่วน
ควบคุม: A_nice0	0	19,068	1,271	1.00
ควบคุม: B_nice0	0	19,119	1,275	1.00
ทดลอง: HIGH_nice0	0	29,924	1,995	64.4
ทดลอง: LOW_nice19	19	465	31	1.00
รันตัวเดียว: renice_target	15	30,658	2,555	—

- nice เท่ากัน scheduler แบ่ง CPU ให้เท่ากันจริง และ nice ต่างกันสร้างความต่างมหาศาล: กรณีควบคุมได้ 19,068 กับ 19,119 รอบ ต่างกันเพียง 0.27% สอดคล้องกับที่ ps รายงาน %CPU ที่ 55.3 กับ 55.5 พอเปลี่ยนค่า nice ของตัวหนึ่งเป็น 19 ผลคือ 29,924 รอบ เทียบกับ 465 รอบ หรือ **ประมาณ 64 เท่า** ค่า %CPU จาก ps (106 เทียบกับ 1.7) และ TIME+ จาก top (0:04.48 เทียบกับ 0:00.07) ก็ชี้ไปทางเดียวกัน

- **nice มีความหมายเฉพาะตอนที่มีการแย่ง CPU:** เป็นข้อสรุปที่ตัวเลขในตาราง แสดงชัดที่สุด `renice_target` ถูกตั้งเป็น `nice 15` ซึ่งถือว่าใจดีมาก แต่เพราะรันอยู่ตัวเดียว จึงทำได้ถึง 2,555 รอบต่อวินาที ซึ่ง **สูงที่สุดในทุกการทดลอง** สูงกว่า `HIGH_nice0` ที่ `nice` เป็น 0 เสียอีก และเมื่อรวมผลงานของสอง process ในกรณีควบคุมเข้าด้วยกันจะได้ $1,271 + 1,275 = 2,546$ รอบต่อวินาที ซึ่งเท่ากับกรณีที่มี process เดียวครองแกนนั้นทั้งแกน (ต่างกัน 0.4%) แปลว่า CPU หนึ่งแกนถูกใช้จนเต็ม ในทุกกรณี เพียงแต่ถูกแบ่งกันคนละแบบ `nice` จึงกำหนดเพียง **ลำดับ** ในการแย่ง ไม่ใช่โควตา
- **ความสัมพันธ์ของคอลัมน์ NI, PRI และ PR:** จากผลการทดลอง PRI ของ `ps` เท่ากับ $19 - NI$ (`nice 0` ได้ 19, `nice 15` ได้ 4, `nice 19` ได้ 0) ส่วน PR ของ `top` เท่ากับ $20 + NI$ (`nice 0` ได้ 20, `nice 19` ได้ 39) คือแสดงข้อมูล ชุดเดียวกันแต่ “กลับด้าน” กัน ของ `ps` ยิ่งมากยิ่งขึ้นสำคัญ ส่วนของ `top` เป็นเลข priority ดิบของเคอร์เนลซึ่งยิ่งน้อยยิ่งสำคัญ นอกจากนี้ STAT ของ process ที่ `nice` มากกว่า 0 จะมีตัว N เพิ่มมา (RN+, SN+) ใช้ตรวจสอบอย่างรวดเร็วได้ว่ามีใครถูก `renice` ไว้บ้าง
- **สิทธิ์เป็นแบบทางเดียว:** ผู้ใช้ทั่วไป “สละสิทธิ์” ได้เสมอ (`renice +8` สำเร็จ) แต่ “ทวงคืน” ไม่ได้เลย (`renice 0` ล้มเหลว ทั้งที่เป็น process ของตัวเองแท้ ๆ) เป็นการป้องกันไม่ให้ผู้ใช้คนใดตั้งค่าติดลบ เพื่อยึด CPU ทั้งเครื่อง อีกจุดที่น่าสนใจคือ `nice -n -5` แม้ตั้งค่าไม่สำเร็จ แต่ยัง **รันโปรแกรมต่อ** ด้วยค่าเดิม ไม่ได้ยกเลิกการทำงานทิ้ง

1.3 การ kill process และ job

ชื่อคำสั่ง `kill` ทำให้เข้าใจผิดได้ง่าย เพราะหน้าที่จริงคือ **ส่งสัญญาณ (signal) ไปให้ process** ไม่ใช่การฆ่าเสมอไป สัญญาณบางตัวสั่งให้หยุดชั่วคราว บางตัวสั่งให้ทำงานต่อ ที่คนเข้าใจว่า `kill` คือการฆ่าเป็นเพราะสัญญาณ **ตั้งต้น** คือ `SIGTERM` ซึ่งมีพฤติกรรม โดยปริยายคือให้ process จบการทำงาน

Table 3: สัญญาณที่ใช้อยู่ และวิธีระบุเป้าหมาย

สัญญาณ	พฤติกรรมโดยปริยาย	รูปแบบคำสั่ง	เป้าหมาย
1 SIGHUP	จบ ใช้เมื่อ terminal ถูกปิด แต่ daemon มักตีความว่าโหลด config ใหม่	<code>kill 1234</code>	<code>SIGTERM</code> ให้ PID 1234
2 SIGINT	จบ คือสัญญาณจากการกด CTRL+C	<code>kill -9 1234</code>	<code>SIGKILL</code> (ระบุ ด้วย เลข หรือชื่อก็ได้)
3 SIGQUIT	จบพร้อมสร้าง core dump	<code>kill %1</code>	job หมายเลข 1 ของ shell นี้
9 SIGKILL	จบทันที ดักจับไม่ได้	<code>kill -- -1234</code>	ทุก process ใน process group
15 SIGTERM	ขอให้จบ (ค่าตั้งต้นของ <code>kill</code>)	<code>pkill -f "pattern"</code>	ทุกตัวที่ command line เดิม ตรงกับรูปแบบ
18 SIGCONT	สั่งให้ทำงานต่อ	<code>killall progname</code>	ทุกตัวที่ ชื่อโปรแกรม ตรงกันแบบตรงตัว
19 SIGSTOP	หยุดชั่วคราว ดักจับไม่ได้	<code>kill -0 1234</code>	ไม่ส่งจริง แค่ตรวจว่ายังมีอยู่ไหม
20 SIGTSTP	หยุด ชั่วคราว จาก การ กด CTRL+Z		

1.3.1 ผลการทดลอง

เพื่อพิสูจน์ว่า `SIGTERM` ดักจับได้แต่ `SIGKILL` ดักจับไม่ได้ ได้เขียนโปรแกรม `q1_sigdemo.c` ที่ลงทะเบียน handler ไว้กับ `SIGTERM` และ `SIGINT` ด้วย `signal()` แล้ววนเรียก `pause()` รอสัญญาณ เมื่อสัญญาณมาถึง handler จะพิมพ์ข้อความด้วย `write()` (ซึ่งเป็น `async-signal-safe` ต่างจาก `printf()`) แล้ว `pause()` ก็วนรอต่อ process จึงไม่มีวันจบด้วยสองสัญญาณนี้

```
$ kill -l
1) SIGHUP  2) SIGINT  3) SIGQUIT  4) SIGILL  5) SIGTRAP
6) SIGABRT 7) SIGBUS  8) SIGFPE  9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
...

$ sleep 300 &          -> pid 5790
  PID    PPID STAT COMMAND
  5790    5689 S+   sleep
$ kill 5790
$ ps -o pid,stat,comm -p 5790      # no data row = the process is gone
  PID STAT COMMAND
exit status of ps = 1
```

Code 9: kill -l (ตามมา 4 แถวแรก) และ kill ด้วยสัญญาณที่ตั้ง

```
[sigdemo] pid = 5794, catching SIGTERM(15) and SIGINT(2)
$ kill 5794          # SIGTERM -> the handler runs, the process survives
[sigdemo] caught SIGTERM (15) -- handler ran, still alive
$ kill -INT 5794     # SIGINT -> the handler runs, the process survives
[sigdemo] caught SIGINT (2) -- handler ran, still alive
$ ps -o pid,stat,comm -p 5794
  PID STAT COMMAND
  5794 S+   q1_sigdemo
$ kill -9 5794       # SIGKILL can be neither caught nor ignored
bash: 5794 Killed    /tmp/ps03_q1/q1_sigdemo
$ ps -o pid,stat,comm -p 5794
  PID STAT COMMAND
exit status of ps = 1
```

Code 10: process ที่ติดกับ SIGTERM ไว้ ต้องใช้ kill -9 เท่านั้น

```
$ ps -o pid,stat,comm -p 5803      # S = sleeping
  PID STAT COMMAND
  5803 S+   sleep
$ kill -STOP 5803
$ ps -o pid,stat,comm -p 5803      # T = stopped
  PID STAT COMMAND
  5803 T+   sleep
$ kill -CONT 5803
$ ps -o pid,stat,comm -p 5803      # back to S again
  PID STAT COMMAND
  5803 S+   sleep
```

Code 11: kill -STOP / -CONT พักและปลุกแทนการฆ่า

```
$ jobs -l
[1] 5813 Running          sleep 400 &
[2]- 5814 Running          sleep 500 &
[3]+ 5815 Running          sleep 600 &
$ kill %2                  # send SIGTERM to job number 2
[2]- Terminated          sleep 500
$ jobs -l
[1]- 5813 Running          sleep 400 &
[3]+ 5815 Running          sleep 600 &
$ kill -9 %1 %3            # kill the two remaining jobs
bash: 5813 Killed          sleep 400
bash: 5815 Killed          sleep 600
$ jobs -l                  # no output = no job left
```

Code 12: การฆ่า job ด้วย job spec %n

```
$ pgrep -a sleep
5822 sleep 700
5823 sleep 700
5824 sleep 700
$ pkill -f 'sleep 700'     # kill every process whose full command line matches
[1] Terminated          sleep 700
[2]- Terminated          sleep 700
[3]+ Terminated          sleep 700
$ pgrep -a sleep
exit status of pgrep = 1  (1 = nothing matched)
```



```
$ pgrep -a q1_sigdemo
5830 /tmp/ps03_q1/q1_sigdemo
5831 /tmp/ps03_q1/q1_sigdemo
$ killall -9 q1_sigdemo      # kill by exact program name
$ pgrep -a q1_sigdemo
exit status of pgrep = 1

$ kill -0 5837  -> pid 5837 is alive (exit status 0)
$ kill -0 5837  -> bash: kill: (5837) - No such process (exit status 1)
```

Code 13: pgrep / pkill / killall อ้างด้วยชื่อ และ kill -0 ตรวจว่ายังอยู่ไหม

1.3.2 อภิปรายผล

- **SIGTERM** คือการ “ขอ” ส่วน **SIGKILL** คือการ “บังคับ”: q1_sigdemo เลือกที่จะพิมพ์ข้อความแล้วอยู่ต่อ ทำให้ kill และ kill -INT ไม่มีผลใด ๆ ต่อการมีอยู่ของ process จนต้องใช้ kill -9 ซึ่งเคอร์เนลจัดการให้เอง โปรแกรมไม่มีโอกาสแม้แต่จะรู้ตัว ในทางปฏิบัติจึงควรใช้ kill ก่อนเสมอ เพราะ SIGTERM เปิดโอกาสให้โปรแกรมเก็บกวาดก่อนจบ เช่นเขียน buffer ลงไฟล์ ปิด socket หรือปลด lock ส่วน kill -9 ตัดจบทันทีทำให้ไฟล์ชั่วคราวค้างและ process ลูกร้างกลายเป็น orphan จึงควรใช้เป็นทางเลือกสุดท้ายหลังรอสัก 5-10 วินาที
- **SIGSTOP** ไม่ใช่การฆ่าและไม่ใช่ว่าการหยุดถาวร: process เปลี่ยนสถานะ จาก s เป็น t แล้วกลับมาเป็น s ได้ด้วย SIGCONT ระหว่างที่เป็น t process ยังอยู่ในระบบและยังจองหน่วยความจำเหมือนเดิม เพียงแต่ scheduler จะไม่จัด CPU ให้ ประโยชน์จริงคือใช้แข่งงานหนักไว้ชั่วคราว แล้วค่อยปลุกกลับมาโดยไม่ต้องเริ่มงานใหม่ (สัญญาณคู่นี้คือกดปุ่มเบรคหลัง CTRL+Z, fg และ bg ที่ทดลองต่อในข้อ 2)
- **%n** เป็นของ shell ไม่ใช่ของเคอร์เนล: เคอร์เนลรู้จักแค่ PID เลข job [1] [2] [3] เป็นสิ่งที่ shell จัดไว้เองสำหรับ process ที่ตัวเองสร้าง เวลาสั่ง kill %2 shell จะแปลงเป็น PID แล้วค่อยส่งสัญญาณ ผลที่ตามมาคือ job spec ใช้ได้เฉพาะใน shell ที่เป็นเจ้าของ job นั้น ถ้าเปิด terminal ใหม่ ต้องกลับไปใช้ PID แทน ข้อดีคือสะดวกและอาจได้หลายตัวพร้อมกันดังที่ทดลองด้วย kill -9 %1 %3
- **pkill -f** สะดวกแต่ต้องระวังมาก: option -f จับคู่กับ *command line* เต็ม ซึ่งกว้างกว่าที่คิด ระหว่างเก็บผลการทดลองนี้เกิดปัญหาจริง คือสคริปต์ที่สั่ง pkill -f 'sleep 700' ได้ฆ่า ตัวเอง ไปด้วย เพราะ command line ของสคริปต์มีข้อความนั้นประกอบอยู่ ทำให้ผลการทดลองขาดไปครึ่งหนึ่ง แนวปฏิบัติที่ปลอดภัยคือสั่ง pgrep ด้วยเงื่อนไขเดียวกันดูก่อนเสมอ แล้วจึงเปลี่ยนเป็น pkill ส่วน killall จับคู่กับชื่อโปรแกรมแบบตรงตัว จึงปลอดภัยกว่าแต่ยืดหยุ่นน้อยกว่า และต้องระวังว่าบน UNIX บางตระกูล (เช่น Solaris) คำสั่งชื่อเดียวกันนี้หมายถึงการฆ่าทุก process ในระบบ
- **เรื่องสิทธิ์และข้อยกเว้น**: ผู้ใช้ทั่วไปส่งสัญญาณได้เฉพาะ process ของตัวเอง และแม้แต่ root ก็ฆ่า process ที่ติดอยู่ในสถานะ d (uninterruptible sleep เช่นค้างรอ I/O) ด้วย kill -9 ไม่ได้ เพราะสัญญาณจะถูกพักไว้จนกว่า process จะออกจากสถานะนั้น ส่วน kill -0 เป็นสำนวนมาตรฐานในสคริปต์ที่ใช้ถามว่า PID ยังอยู่ไหม โดยไม่ก่อผลข้างเคียง มักใช้คู่กับการรอหลังส่ง SIGTERM ก่อนตัดสินใจว่าต้องใช้ SIGKILL หรือไม่

2 ข้อ 2: การรัน shell script แบบ foreground / background และ job control

2.1 สถานการณ์ที่ 1: ./ss1_1.sh เทียบกับ ./ss1_1.sh &

ตามคำแนะนำท้ายโจทย์ ได้เขียน ss1_1.sh เป็น for loop ที่เรียก sleep 1 รอบละหนึ่งวินาทีรวมสิบรอบ แทนการเรียก sleep 10 ครั้งเดียว เพื่อให้เห็น ความคืบหน้าระหว่างที่สคริปต์กำลังทำงาน

```
1 #!/bin/bash
2 # CPE 333 PS03 - Q2, first situation
3 # Calls "sleep" for a total of 10 seconds, one second per iteration,
4 # so the progress can be watched while the script is running.
5
```

```

6 echo "[ss1_1] start pid=$$ at $(date +%T)"
7
8 for i in $(seq 1 10); do
9     sleep 1
10    echo "[ss1_1] tick $i/10"
11 done
12
13 echo "[ss1_1] done pid=$$ at $(date +%T)"

```

Code 14: ss1_1.sh

ไฟล์ที่เพิ่งสร้างใหม่ได้สิทธิ์ rw-r--r-- (644) ตามค่า umask มาตรฐาน คือไม่มีบิต x เคอร์เนลจึงปฏิเสธที่จะ exec ไฟล์นั้น ต้องเพิ่มบิต x ด้วย chmod ก่อน (อีกวิธีที่ใช้ได้โดยไม่ต้อง chmod คือสั่ง bash ss1_1.sh เพราะเป็นการให้ bash อ่านไฟล์เป็นข้อมูล ไม่ใช่ให้เคอร์เนล exec ตัวไฟล์ จึงต้องการเพียงสิทธิ์อ่าน)

```

student@ubuntu:~/PS03$ ls -l ss1_1.sh

-rw-r--r-- 1 poomipat poomipat 345 Aug 31 15:23 ss1_1.sh
student@ubuntu:~/PS03$ ./ss1_1.sh

bash: ./ss1_1.sh: Permission denied
student@ubuntu:~/PS03$ chmod +x ss1_1.sh

student@ubuntu:~/PS03$ ls -l ss1_1.sh

-rwxr-xr-x 1 poomipat poomipat 345 Aug 31 15:23 ss1_1.sh

```

Code 15: Permission denied และการแก้ด้วย chmod

```

student@ubuntu:~/PS03$ date +%T ; time ./ss1_1.sh ; date +%T

15:23:40
[ss1_1] start pid=6860 at 15:23:40
[ss1_1] tick 1/10
[ss1_1] tick 2/10
[ss1_1] tick 3/10 ... tick 9/10          (7 lines omitted)
[ss1_1] tick 10/10
[ss1_1] done pid=6860 at 15:23:50

real    0m10.030s
user    0m0.019s
sys     0m0.019s
15:23:50

student@ubuntu:~/PS03$ cat /tmp/ps03_q2/fg1.txt

  PID    PPID    PGID    TPGID  STAT  COMMAND
 6844    6841    6844    6860  Ss    bash --norc -i
 6856    6844    6856    6860  S      bash --norc -i
 6860    6844    6860    6860  S+     /bin/bash ./ss1_1.sh
 6866    6860    6860    6860  S+     sleep 1
 6867    6856    6856    6860  R      ps -o pid,ppid,pgid,tpgid,stat,args

```

Code 16: รันแบบ foreground – terminal ถูกยึดไว้จนกว่าจะจบ พร้อมภาพจาก ps ที่สั่งถ่ายไว้ล่วงหน้า

```

student@ubuntu:~/PS03$ date +%T ; ./ss1_1.sh & date +%T

15:23:54
[1] 6880
15:23:54
student@ubuntu:~/PS03$ [ss1_1] start pid=6880 at 15:23:54
jobs -l

[1]+  6880 Running                  ./ss1_1.sh &
student@ubuntu:~/PS03$ ps -o pid,ppid,pgid,tpgid,stat,args

  PID    PPID    PGID    TPGID  STAT  COMMAND
 6844    6841    6844    6887  Ss    bash --norc -i
 6880    6844    6880    6887  S      /bin/bash ./ss1_1.sh
 6884    6880    6880    6887  S      sleep 1
 6887    6844    6887    6887  R+     ps -o pid,ppid,pgid,tpgid,stat,args
[ss1_1] tick 1/10

```

```
student@ubuntu:~/PS03$ wait
[ss1_1] tick 2/10 ... tick 10/10          (9 lines omitted)
[ss1_1] done   pid=6880 at 15:24:04
[1]+ Done                      ./ss1_1.sh
student@ubuntu:~/PS03$ jobs -l
```

Code 17: รันแบบ background – prompt กลับมาทันที

2.1.1 อภิปรายผล

Table 4: สรุปความแตกต่างของการรันแบบ foreground และ background

ประเด็น	./ss1_1.sh	./ss1_1.sh &
prompt กลับเมื่อใด	หลังสคริปต์จบ วัดได้ real 0m10.030s	ทันที date ก่อนและหลังได้ 15:23:54 เท่ากัน
เจ้าของ terminal	สคริปต์ TPGID เท่ากับ PGID ของตัวเอง และ STAT มี +	shell ยังเป็นเจ้าของ STAT ของสคริปต์ไม่มี +
สถานะเป็น job	ไม่ถูกจัดเป็น job	ถูกจัดเป็น [1] 6880 และแจ้ง [1]+ Done เมื่อจบ
พิมพ์คำสั่งอื่น	ไม่ได้ ทุกตัวอักษรจะเข้าคิวรอ	ได้ทันที ทดสอบด้วย jobs และ ps ระหว่างที่ job ยังรันอยู่
สัญญาณคีย์บอร์ด	จาก CTRL+C / CTRL+Z ส่งถึงสคริปต์	ส่งไม่ถึง เพราะไม่ได้อยู่ใน foreground process group
stdout	terminal เดียวกัน	terminal เดียวกัน จึงปนกับ prompt
การรอให้จบ	shell รออยู่แล้วอัตโนมัติ	ต้องสั่ง wait เอง หรือดูจาก jobs

- ความแตกต่างที่แท้จริงคือ “ใครเป็นเจ้าของ terminal” ไม่ใช่แค่รอหรือไม่รอ: ผลจาก ps ชี้ชัดว่าตอนรันแบบ foreground คอลัมน์ TPGID ของทุก process มีค่าเป็น 6860 ซึ่งเป็น PGID ของ ss1_1.sh เอง แปลว่าเคอร์เนลยกสิทธิ์ควบคุม terminal ให้ process group ของสคริปต์ไปแล้ว และ STAT ของสคริปต์กับ sleep ลูกของมัน มีเครื่องหมาย + ต่อท้าย (s+) ซึ่ง ps ใช้บอกว่า “อยู่ใน foreground process group” ในทางกลับกันตอนรันแบบ background TPGID ไม่ใช่ของสคริปต์และ STAT เป็น s เฉย ๆ นอกจากนั้นบรรทัด 6844 ... 6860 ss ยังแสดงว่า shell ไม่ได้หยุดทำงาน แต่ถูกบล็อกอยู่ใน wait() รอให้ลูกจบ ซึ่งไม่กิน CPU
- background ไม่ได้แยก output ออกไป: เป็นจุดที่เข้าใจผิดบ่อย ผลการทดลองแสดงชัดว่าข้อความ [ss1_1] start pid=6880 โผล่ต่อท้าย prompt และ [ss1_1] tick 1/10 แทรกกลางผลลัพธ์ของ ps เพราะ job เบื้องหลังยังเขียน stdout ลง terminal เดิม ถ้าไม่อยากให้ปนกัน ต้อง redirect เอง เช่น ./ss1_1.sh > out.log 2>&1 &
- & ทำให้ shell จัดงานนั้นเป็น job: shell พิมพ์ [1] 6880 คือเลข job กับ PID ทำให้อ้างด้วย %1 ได้ภายหลัง และเมื่องานจบจะแจ้ง [1]+ Done ที่ prompt ถัดไป ส่วน exit status ต้องเก็บผ่าน wait %1 หรือ \$? เพราะไม่มี \$? ให้ดูทันทีเหมือน foreground ข้อควรระวังคือ job ที่รันด้วย & ยังผูกกับ terminal เดิม เมื่อปิด terminal เคอร์เนลจะส่ง SIGHUP ทำให้งานตายตามไปด้วย ถ้าต้องการให้อยู่รอดต้องใช้ nohup หรือ disown

2.2 สถานการณ์ที่ 2: CTRL+Z, jobs, fg และ bg

ss1_2.sh เรียก sleep 1000 เพื่อให้สคริปต์อยู่นานพอที่จะหยุดด้วย CTRL+Z แล้วปลุกกลับมาด้วย fg หรือ bg

```
1 #!/bin/bash
2 # CPE 333 PS03 - Q2, second situation
3 # Calls "sleep" with 1000 seconds, so the script stays alive long enough
4 # to be suspended with CTRL+Z and resumed again with fg / bg.
5
6 echo "[ss1_2] start  pid=$$ at $(date +%T)"
7
8 sleep 1000
9
10 echo "[ss1_2] done  pid=$$ at $(date +%T)"
```

Code 18: ss1_2.sh

```

student@ubuntu:~/PS03$ ./ss1_2.sh

[ss1_2] start pid=6914 at 15:24:14
^Z
[1]+  Stopped                  ./ss1_2.sh
student@ubuntu:~/PS03$ jobs -l

[1]+  6914 Stopped              ./ss1_2.sh
student@ubuntu:~/PS03$ ps -o pid,ppid,pgid,tpgid,stat,args

  PID   PPID   PGID   TPGID STAT COMMAND
  6908   6905   6908   6920 Ss  bash --norc -i
  6914   6908   6914   6920 T   /bin/bash ./ss1_2.sh
  6916   6914   6914   6920 T   sleep 1000
  6920   6908   6920   6920 R+  ps -o pid,ppid,pgid,tpgid,stat,args

student@ubuntu:~/PS03$ bg %1

[1]+ ./ss1_2.sh &
student@ubuntu:~/PS03$ jobs -l

[1]+  6914 Running              ./ss1_2.sh &
student@ubuntu:~/PS03$ ps -o pid,ppid,pgid,tpgid,stat,args

  PID   PPID   PGID   TPGID STAT COMMAND
  6908   6905   6908   6924 Ss  bash --norc -i
  6914   6908   6914   6924 S   /bin/bash ./ss1_2.sh
  6916   6914   6914   6924 S   sleep 1000

```

Code 19: CTRL+Z ทำให้ทั้ง process group หยุด (สถานะ T) แล้ว bg ปลุกกลับมาทำงานเบื้องหลัง

```

student@ubuntu:~/PS03$ fg %1

./ss1_2.sh
^Z
[1]+  Stopped                  ./ss1_2.sh
[2]  Done                      ( sleep 2; ps -o pid,ppid,pgid,tpgid,stat,args > /tmp/ps03_q2/fg2.txt 2>&1 )
student@ubuntu:~/PS03$ jobs -l

[1]+  6914 Stopped              ./ss1_2.sh
student@ubuntu:~/PS03$ cat /tmp/ps03_q2/fg2.txt

  PID   PPID   PGID   TPGID STAT COMMAND
  6908   6905   6908   6914 Ss  bash --norc -i
  6914   6908   6914   6914 S+  /bin/bash ./ss1_2.sh
  6916   6914   6914   6914 S+  sleep 1000
  6927   6908   6927   6914 S   bash --norc -i
  6930   6927   6927   6914 R   ps -o pid,ppid,pgid,tpgid,stat,args

student@ubuntu:~/PS03$ kill %1

[1]+  Stopped                  ./ss1_2.sh
student@ubuntu:~/PS03$ jobs -l

[1]+  6914 Terminated          ./ss1_2.sh
student@ubuntu:~/PS03$ jobs -l

```

Code 20: fg ดึงกลับมาเป็น foreground และคืนสิทธิ์ terminal ให้ job แล้วปิดงานด้วย kill %1

การทดลองเพิ่มเติมด้านล่างแสดงกรณีที่ bg ใช้ไม่ได้ และต้องใช้ fg เท่านั้น คือ job ที่ต้องอ่านข้อมูลจากคีย์บอร์ด

```

student@ubuntu:~/PS03$ ( read -p "type your name: " n ; echo "[bgread] you typed: $n" ) &

[1] 6939
student@ubuntu:~/PS03$ type your name: jobs -l

[1]+ 6939 Stopped (tty input)    ( read -p "type your name: " n; echo "[bgread] you typed: $n" )
student@ubuntu:~/PS03$ bg %1

```

```
[1]+ ( read -p "type your name: " n; echo "[bgread] you typed: $n" ) &
student@ubuntu:~/PS03$ jobs -l

[1]+  6939 Stopped (tty input)      ( read -p "type your name: " n; echo "[bgread] you typed: $n" )
student@ubuntu:~/PS03$ fg %1

( read -p "type your name: " n; echo "[bgread] you typed: $n" )
CPE333
[bgread] you typed: CPE333
student@ubuntu:~/PS03$ jobs -l
```

Code 21: job ที่อ่าน stdin จะถูกหยุดเองด้วย SIGTTIN

2.2.1 อภิปรายผล

Table 5: กลไกเบื้องหลัง CTRL+Z, bg และ fg

การกระทำ	สัญญาณที่ส่ง	สถานะที่เปลี่ยน	ใครคุม terminal ต่อ
CTRL+Z	SIGTSTP (20)	s+ → T ทั้ง process group	shell ทาง terminal คืน prompt กลับมา
bg %1	SIGCONT (18)	T → s (ไม่มี +)	shell ยังคงอยู่เหมือนเดิม prompt ใช้ได้ทันที
fg %1	SIGCONT (18)	T → s+	ยกให้ job แล้วรอจน job จบหรือถูกหยุดอีก

- **CTRL+Z** คือการ “พัก” ไม่ใช่การ “ฆ่า” และส่งถึงทั้ง process group: ps ยืนยันว่า process ยังอยู่ครบ เพียงเปลี่ยนสถานะจาก s+ เป็น T ต่างจาก CTRL+C ที่ส่ง SIGINT แล้วจบไปเลย ที่สำคัญคือทั้ง 6914 /bin/bash ./ss1_2.sh และลูกของมันคือ 6916 sleep 1000 เปลี่ยนเป็น T พร้อมกันทั้งที่เกิดเพียงครั้งเดียว เพราะทั้งคู่มี PGID เดียวกันคือ 6914 และ terminal ส่งสัญญาณให้ทั้ง foreground process group หากเคอร์เนลส่งให้เฉพาะตัว bash ของสคริปต์ ตัว sleep ก็จะไม่จบเวลาต่อไป ซึ่งไม่ใช่สิ่งที่ผู้ใช้คาดหวัง
- **ความต่างของ fg กับ bg อยู่ที่สิทธิ์ควบคุม terminal ไม่ใช่ที่การปลุก:** ทั้งสองคำสั่งส่ง SIGCONT เหมือนกันทุกประการ job จึงกลับมาทำงานต่อได้ทั้งคู่ (สถานะเปลี่ยนจาก T เป็น s ทั้งสองกรณี) สิ่งที่ต่างคือ bg ปลอຍให้ shell ถือ terminal ไว้เหมือนเดิม prompt จึงกลับมาใช้ทันที ส่วน fg เรียก tcsetpgrp() ยกสิทธิ์ควบคุม terminal ให้ process group ของ job แล้ว shell ก็เข้าไปรอ หลักฐานคือภาพจาก ps ตอนที่ job อยู่ใน foreground มี TPGID เป็น 6914 และ STAT เป็น s+ เทียบกับตอน bg ที่เป็น s เฉย ๆ นอกจากนี้ jobs ยังเป็นตารางของ shell เอง ไม่ใช่ของเคอร์เนล โดย + หมายถึง current job (ตัวที่ fg หรือ bg จะเลือกถ้าไม่ระบุ %n) และ - คือ previous job
- **มีงานบางประเภทที่ bg ใช้ไม่ได้เลย:** การทดลองสุดท้ายแสดงชัดที่สุด job ที่เรียก read เพื่ออ่านคีย์บอร์ดถูกเคอร์เนลหยุดทันทีด้วย SIGTTIN และ jobs รายงานเหตุผลตรง ๆ ว่า Stopped (tty input) ที่สำคัญคือเมื่อสั่ง bg %1 job กลับไปหยุดที่เดิมอีกครั้งด้วยเหตุผลเดียวกัน เพราะ bg ให้แค่ SIGCONT ไม่ได้ให้สิทธิ์อ่าน terminal ต้องสั่ง fg %1 เท่านั้น job จึงอ่านข้อความ CPE333 ที่พิมพ์เข้าไปได้และทำงานจนจบ สรุปเป็นหลักการได้ว่า bg เหมาะกับงานที่คำนวณอย่างเดียว ส่วนงานที่ต้องโต้ตอบกับผู้ใช้ต้องอยู่ใน foreground เสมอ
- **การ kill job ที่ถูกหยุดอยู่ และการเชื่อมโยงกลับไปข้อ 1:** SIGTERM เพียงอย่างเดียวจะค้างรออยู่เฉย ๆ เพราะ process ในสถานะ T ไม่ได้ถูกจัดให้ทำงานจึงยังไม่ประมวลผลสัญญาณ แต่ผลการทดลองแสดงว่า job ตายทันที เพราะ kill ของ bash ส่ง SIGCONT ตามไปให้ด้วย เมื่อรู้ว่าเป้าหมายถูกหยุดอยู่ สังเกตได้จาก jobs -l ครั้งแรก ที่รายงาน Terminated และครั้งที่สองไม่เหลืออะไร เพราะ shell แจ้งสถานะสุดท้าย เพียงครั้งเดียวแล้วลบทิ้ง โดยรวมแล้ว fg และ bg คือหน้าากที่สวยงามของ kill -CONT ที่ทดลองไว้ในข้อ 1 โดย bg %1 ให้ผลเหมือน kill -CONT <PID> ทุกประการ ส่วน fg %1 คือ kill -CONT บวกกับการยก terminal ให้

3 ข้อ 5: ฟังก์ชันภาษาซีจำลองการทำงานของ STCF

STCF (Shortest Time-to-Completion First) หรือ SRTF เป็นนโยบาย จัดตารางแบบ *preemptive* ที่ทุกหน่วยเวลา จะมอบ CPU ให้ process ซึ่ง **เหลืองานน้อยที่สุด** ต่างจาก SJF ที่เมื่อเริ่มแล้วต้องปล่อยให้ทำงานจน งานของข้อนี้คือ เขียนฟังก์ชัน `simulate_stfc()` เติมลงในไฟล์ `PS3.c` ที่โจทย์เตรียมไว้ ส่วน `main()` ใช้ของเดิมเกือบทั้งหมด แค่เพียง จุดเดียว ตามที่อธิบายท้ายหัวข้อย่อย Source code (หัวข้อ 3.1)

Table 6: ข้อกำหนดจากโจทย์และการแปลงเป็นเงื่อนไขในโปรแกรม

ข้อกำหนด	การนำไปใช้ในโค้ด
ฟังก์ชันและ context switch เร็วมาก ไม่ต้องนับ เป็นการใช้ CPU	ทุกหน่วยเวลาถูกใช้ทำงานจริงเสมอ ไม่มีช่องว่างแทรก ระหว่างช่วง
process ที่มาถึงเวลา T หมายถึงมาถึงตอนท้าย ของเวลา $T - 1$ แต่ก่อนที่ scheduler จะอัปเดต ready queue ของเวลา T	ที่เวลา T process นั้นอยู่ใน ready queue แล้ว เงื่อนไข จึงเป็น <code>arrival <= time</code> (ไม่ใช่ <code><</code>)
มาถึงพร้อมกันหลายตัว ให้เรียงตาม <code>process_id</code>	เทียบ <code>pid</code> จากน้อยไปมาก
เวลาที่เหลือเท่ากัน ให้เรียงตาม <code>process_id</code>	เทียบ <code>pid</code> จากน้อยไปมาก

เงื่อนไขสองข้อสุดท้ายยุบรวมเป็นกฎเดียวได้ เพราะ process ที่เพิ่งมาถึงพร้อมกัน ยังไม่ได้ใช้ CPU เลย เวลาที่เหลือ จึงเท่ากับ `burst` เดิม ๆ กรณี “มาถึงพร้อมกัน” จึงเป็นกรณีเฉพาะของ “เวลาที่เหลือเท่ากัน” โค้ดจึงเทียบ `pid` เพียง จุดเดียว ส่วน **รูปแบบผลลัพธ์** นั้นตัวอย่างในโจทย์ ถูกจัดรูปแบบไม่สมบูรณ์ (บางบรรทัดขึ้นบรรทัดใหม่คั่นระหว่าง [0-3]: กับชื่อ process) แต่สองบรรทัดสุดท้ายที่สมบูรณ์คือ [8-11]: P3 แสดงว่ารูปแบบที่ถูกต้องคือหนึ่งบรรทัด ต่อหนึ่งช่วงเวลา โปรแกรมนี้จึงพิมพ์ตามนั้น โดยรวมหน่วยเวลาที่ติดกันและเป็นของ process เดียวกันเข้าเป็นช่วง เดียว และช่วงที่ไม่มี process ใดพร้อมทำงานแสดงเป็น IDLE ตามตัวอย่างในโจทย์

3.1 Source code

```

1  /* Print one merged time slot. "idx" is the index of the process that owned
2  * the CPU during [start, end), or -1 when the CPU was idle. */
3  static void print_slot(int start, int end, int idx, const Process *procs)
4  {
5      if (start >= end)
6          return;                      /* empty slot, before the first decision */
7
8      if (idx < 0)
9          printf("[%d-%d]: IDLE\n", start, end);
10     else
11         printf("[%d-%d]: P%d\n", start, end, procs[idx].pid);
12 }
13
14 void simulate_stfc(Process *procs, int n)
15 {
16     int time = 0;
17     int done = 0;
18
19     int current = -1; /* index of the process on the CPU, -1 = idle */
20     int slot_start = 0; /* time at which the current slot began */
21
22     /* A process with no work to do would never reach remaining == 0 by running,
23      * so retire it up front instead of letting it stall the simulation. */
24     for (int i = 0; i < n; i++) {
25         if (procs[i].remaining <= 0 && !procs[i].finished) {
26             procs[i].finished = 1;
27             done++;
28         }
29     }
30
31     while (done < n) {
32         /* choose the ready process with the shortest remaining time */
33         int pick = -1;

```

```

34     for (int i = 0; i < n; i++) {
35         if (procs[i].finished || procs[i].arrival > time)
36             continue;
37
38         if (pick == -1 ||
39             procs[i].remaining < procs[pick].remaining ||
40             (procs[i].remaining == procs[pick].remaining &&
41              procs[i].pid < procs[pick].pid))
42             pick = i;
43     }
44
45     /* the owner of the CPU changed, so the previous slot ends here */
46     if (pick != current) {
47         print_slot(slot_start, time, current, procs);
48         slot_start = time;
49         current = pick;
50     }
51
52     if (pick == -1) {
53         time++;          /* nothing has arrived yet, the CPU stays idle */
54         continue;
55     }
56
57     procs[pick].remaining--;
58     time++;
59
60     if (procs[pick].remaining == 0) {
61         procs[pick].finished = 1;
62         done++;
63     }
64 }
65
66 print_slot(slot_start, time, current, procs); /* the last slot */
67 }

```

Code 22: PS3.c – ส่วนที่เขียนเพิ่มเข้าไป

คำอธิบายการทำงาน ลูปนอกเดินทีละหนึ่งหน่วยเวลาจนกว่าจะครบทุก process ซึ่งตรงกับนิยามของ STCF ที่ตัดสินใจใหม่ทุกครั้งทีสถานการณ์เปลี่ยน โดยไม่ต้องแยกกรณีว่าเหตุการณ์ไหนบ้างที่ทำให้ต้องตัดสินใจใหม่ ลูปในเลือกผู้ชนะโดยข้าม process ที่จบแล้วหรือยังมาไม่ถึง แล้วเทียบ remaining หากเท่ากันจึงเทียบ pid ถ้าไม่มีใครพร้อมเลยจะได้ -1 ซึ่งหมายถึง IDLE การรวมช่วงเวลาใช้ตัวแปร current กับ slot_start เมื่อใดที่ผู้ชนะ เปลี่ยนไป แสดงว่าช่วงเดิมจบลงที่เวลาปัจจุบันพอดี จึงพิมพ์ช่วงเดิมแล้วเริ่มนับใหม่ ส่วนการพิมพ์ช่วงสุดท้ายต้องทำนอกลูป เพราะตอนที่ process ตัวสุดท้ายจบ ลูปจะจบทันทีโดยไม่มีการเปลี่ยนผู้ชนะให้ตรวจจับได้อีก

จุดเดียวที่แก้ไขใน main() โค้ดเดิมข้ามบรรทัดหัวตารางด้วย fgets(line, sizeof(line), fp); โดยไม่ดูค่าที่คืนมา ซึ่งถ้าไฟล์ว่างเปล่า ตัวแปร line จะไม่ถูกกำหนดค่า และเมื่อคอมไพล์ด้วย -02 จะได้ warning เพราะ fgets() เป็น warn_unused_result โจทย์เปิดช่องให้แก้ส่วนนี้ได้ (// you may change this if needed) จึงเพิ่ม if (!fgets(...)) { fprintf(stderr, ...); fclose(fp); return 1; } เข้าไป ผลคือคอมไพล์ผ่านโดยไม่มี warning ทั้งที่ -00 และ -02 และไฟล์ว่างจะได้ข้อความแจ้งที่อ่านรู้เรื่อง

3.2 ผลการทดลองและการตรวจสอบความถูกต้อง

Table 7: ข้อมูลเข้าทั้งสามเคส

case1.csv	case2.csv	case3.csv
pid,arrival,burst	pid,arrival,burst	pid,arrival,burst
1,0,7	1,0,2	2,0,5
2,2,4	2,5,3	1,3,2
3,4,1	3,5,3	3,6,1
	4,6,1	

```

PS03 Q5 results
bash-5.2$ export PS1='student@ubuntu:~/PS03/src$ '
student@ubuntu:~/PS03/src$ gcc -Wall -Wextra -o PS3 PS3.c
student@ubuntu:~/PS03/src$ ./PS3 case1.csv
[0-2]: P1
[2-4]: P2
[4-5]: P3
[5-7]: P2
[7-12]: P1
student@ubuntu:~/PS03/src$ ./PS3 case2.csv
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3
student@ubuntu:~/PS03/src$ ./PS3 case3.csv
[0-3]: P2
[3-5]: P1
[5-7]: P2
[7-8]: P3
student@ubuntu:~/PS03/src$

```

Figure 1: screenshot ผลการรันทั้งสามเคสบน terminal จริง (Ubuntu 24.04 บน WSL2)

```

$ ./PS3
Usage: ./PS3 <csvfile>
exit status = 1

$ ./PS3 nosuchfile.csv
Error opening file: No such file or directory
exit status = 1

$ ./PS3 empty.csv          # a completely empty file
Error: empty.csv is empty
exit status = 1

$ ./PS3 header.csv        # header row only, no process at all
exit status = 0 (no process to schedule, so nothing is printed)

```

Code 23: กรณีขอบ: เรียกโปรแกรมผิดวิธีและไฟล์ที่ใช้ไม่ได้

เพื่อให้ไม่เป็นการเชื่อผลลัพธ์ของโปรแกรมตัวเอง ได้ไล่ตารางเวลาของทั้งสามเคสด้วยมือ ก่อน แล้วเก็บคำตอบไว้เป็นไฟล์เฉลย จากนั้นให้สคริปต์เก็บผลสั่ง diff เทียบกับผลลัพธ์จริงของโปรแกรม ซึ่งได้ผลว่าตรงกันทั้งสามเคส (บรรทัด identical, the program agrees with the hand analysis ใน result/q5_verified.txt) ตัวอย่างการไล่ด้วยมือของ case1.csv เป็นดังตารางถัดไป

Table 8: การไล่ตารางเวลาของ case1.csv ทีละหน่วยเวลา

เวลา	process ที่พร้อม (เวลาที่เหลือ)	ผู้ได้ CPU	หมายเหตุ
0-1	P1(7), P1(6)	P1	มีตัวเดียว
2-3	P1(5), P2(4)	P2	P2 มาถึงและเหลือน้อยกว่า จึงแย่ง CPU ไป
4	P1(5), P2(2), P3(1)	P3	P3 มาถึงและเหลือน้อยที่สุด จึงแย่งอีกครั้ง
5-6	P1(5), P2(2)	P2	P3 จบแล้ว
7-11	P1(5)	P1	P2 จบแล้ว เหลือ P1 ตัวเดียว
12	—	—	ทุก process จบหมด

รวมหน่วยเวลาที่ติดกันเข้าด้วยกันจะได้ [0-2]: P1, [2-4]: P2, [4-5]: P3, [5-7]: P2 และ [7-12]: P1 ตรงกับผลลัพธ์ ของโปรแกรมทุกบรรทัด

3.3 อภิปรายผล

Table 9: เปรียบเทียบค่าเฉลี่ยของ STCF กับ FIFO บนข้อมูลชุดเดียวกัน

เคส	STCF			FIFO		
	turnaround	waiting	response	turnaround	waiting	response
case1	6.00	2.00	0.00	8.00	4.00	4.00
case2	3.50	1.25	1.00	4.25	2.00	2.00
case3	3.67	1.00	0.33	3.67	1.00	1.00

- **STCF ให้ average turnaround ต่ำที่สุดจริงตามทฤษฎี:** case1 ได้ 6.00 เทียบกับ 8.00 ของ FIFO และ case2 ได้ 3.50 เทียบกับ 4.25 ส่วน case3 ได้เท่ากันที่ 3.67 ซึ่งไม่ขัดกับทฤษฎี เพราะ STCF รับประกันว่า *ไม่แย่กว่า* ไม่ใช่ว่าต้องดีกว่าเสมอ ในกรณีที่งานสั้นบังเอิญมาถึง ตอนที่ CPU ว่างอยู่แล้ว ลำดับที่ได้จากทั้งสองนโยบายก็จะเหมือนกัน อย่างไรก็ตามแม้ในเคสนั้น STCF ก็ยังชนะเรื่อง response time (0.33 เทียบกับ 1.00)
- **ราคาที่ต้องจ่ายคือ starvation และการต้องรู้ burst ล่วงหน้า:** case1 แสดงชัดว่า P1 มาถึงก่อนใครเพื่อน (เวลา 0) แต่กลับเป็นตัวสุดท้ายที่จบ (เวลา 12) และมีเวลารอสูงสุดคือ 5 หน่วยเวลา เพราะถูกงานที่สั้นกว่าแย่ง CPU ไปสองครั้ง ถ้ามีงานสั้นไหลเข้ามาไม่หยุด process ที่มี burst ยาวก็อาจไม่ได้จบเลย นอกจากนี้การจำลองนี้ทำได้เพราะไฟล์ .csv บอก burst มาให้แล้ว แต่ระบบปฏิบัติการจริงไม่มีทางรู้ล่วงหน้า จึงต้องประมาณจากพฤติกรรมในอดีต หรือเปลี่ยนไปใช้นโยบายที่ไม่ต้องรู้ burst เช่น MLFQ ที่ค่อย ๆ ลดระดับความสำคัญของงานที่ใช้ CPU นาน ซึ่งให้ผลใกล้เคียง STCF โดยไม่ต้องพึ่งข้อมูลอนาคต
- **การแย่ง CPU เกิดเฉพาะตอนที่ process ใหม่มาถึง:** พิสูจน์ได้ง่าย ๆ คือ process ที่กำลังทำงานอยู่มี remaining ลดลงเรื่อย ๆ ส่วนตัวที่รออยู่มีค่าคงที่ ตัวที่กำลังทำงานจึงยังเป็นตัวที่เหลือน้อยที่สุดต่อไปเสมอ เว้นแต่มีตัวใหม่ที่เหลือน้อยกว่าโผล่เข้ามา ผลในเชิงการเขียนโปรแกรมคือ การเลือกผู้ชนะใหม่ทุกหน่วยเวลาแม้จะดูเปลืองแต่ให้คำตอบที่ถูกต้องเสมอ และเขียนง่ายกว่าการไล่จับเหตุการณ์เป็นกรณี ๆ มาก อย่างไรก็ตาม กฎ tie-break ก็ทำให้เกิดการแย่ง CPU ได้ด้วย เห็นได้จาก case3 ที่เวลา 3 ตอนนั้น P2 กำลังทำงานอยู่และเหลืออีก 2 หน่วยพอดีกับที่ P1 มาถึงพร้อม burst 2 หน่วยเท่ากัน ตามกฎของโจทย์ P1 จึงได้ CPU ไปทั้งที่ P2 กำลังทำอยู่ ถ้าใช้กฎอีกแบบที่นิยมกันคือ “เสมอกันให้ตัวที่ทำอยู่ทำต่อ” เพื่อลด context switch ผลของ case3 จะออกมาไม่เหมือนกัน จุดนี้จึงเป็นเหตุผลที่โจทย์ต้องระบุกฎ tie-break ไว้ชัดเจน
- **ช่วง IDLE และประสิทธิภาพ:** case2 มีช่วง [2-5] ที่ P1 จบแล้ว แต่ P2 กับ P3 ยังมาไม่ถึง ในโค้ดจัดการด้วยการเพิ่ม time ที่ละหนึ่ง แล้ววนต่อ ซึ่งตรงกับพฤติกรรมของระบบจริงที่เคอร์เนลรัน idle task หรือสั่งให้ CPU เข้าสู่สถานะประหยัดพลังงาน โค้ดนี้มีความซับซ้อน $O(T \times n)$ เมื่อ T คือเวลารวมและ n คือจำนวน process ซึ่งเพียงพอสำหรับข้อมูลระดับนี้ ถ้าต้องรับข้อมูลขนาดใหญ่จริง ๆ สามารถเปลี่ยนเป็นแบบ event-driven คือคำนวณเฉพาะตอนที่ process มาถึงหรือทำงานจบ แล้วใช้ priority queue เก็บ ready queue จะลดลงเหลือ $O(n \log n)$